

MAE 145: Planning and Estimation

Homework #2

Assigned Jan 15. Due on Jan 22

Notice:

Please upload your solutions in the homework 2 and homework 2 programming sections of grade-scope. The Collaboration Policy for this course is detailed in the syllabus.

Homework exercises:

1. (20 points) Exercise E2.2, from the lecture notes by Bullo and Smith.
2. (18 points) Exercise E2.5 from the lecture notes by Bullo and Smith.
3. (15 points) Programming exercise (tic-tac-toe, part I). Tic-tac-toe (or noughts and crosses) is a simple strategy game in which two players take turns placing a mark on a 3×3 board, attempting to make a row, column, or diagonal of three with their mark. We start with a few exercises (to be continued in a future homework), that serve as a python practice, and where we will use python functions to create a tic-tac-toe simulator.
 - (a) Write a function `create_board()` that creates a tic-tac-toe board with the value of each cell set to the integer 0. *Note:* The function has no input, and its output should use an appropriate numpy array to initialize the tic-tac-toe 3×3 board.
 - (b) Create a function `place(board, player, position)`, where: (a) `player` is the current player (an integer 1 or 2). (b) `position` is a tuple of length 2 specifying a desired location to place the marker, and (c) `board` is as described in part (a). Your function should only allow the current player to place a marker on the board (change the board position to their number) if that position is empty (zero).
 - (c) Here, we will define a function that will determine which positions are available to either player for placing their marker.
Create a function `possibilities(board)` that returns a list of all positions (tuples) on the board that are not occupied (0). (*Hint:* `numpy.where` is a handy function that returns a list of indices that meet a condition.)
Note that `board` is defined as at the end of the part (b).
 - (d) The next step is for the current player to place a marker among the available positions. In this exercise, we will select an available board position at random and place a marker there. Write a function `random_place(board, player)` that places a marker for the current player at random among all the available positions (those currently set to 0). Find possible placements with `possibilities(board)`. Select one possible placement at random using `random.choice(selection)` (to use this, load the `random` module). *Note:* In the template we have used `random.seed(1)` to make the result deterministic and for checking purposes. To make the result truly random, you would comment this line out.

- (e) We will now have both players play consecutively. To do this, create a function called `repeat(n)`, which takes as an input the number of consecutive plays (an integer n , $n \leq 4$) for each player and allows players place n pieces each on `board`. Note that player 1 starts, then 2 follow. You can try with $n = 1, 2, 3$, and 4 moves for each player. For correction purposes, please maintain the `random.seed(1)` in the code. If you would like to see truly random results, you can comment this out.